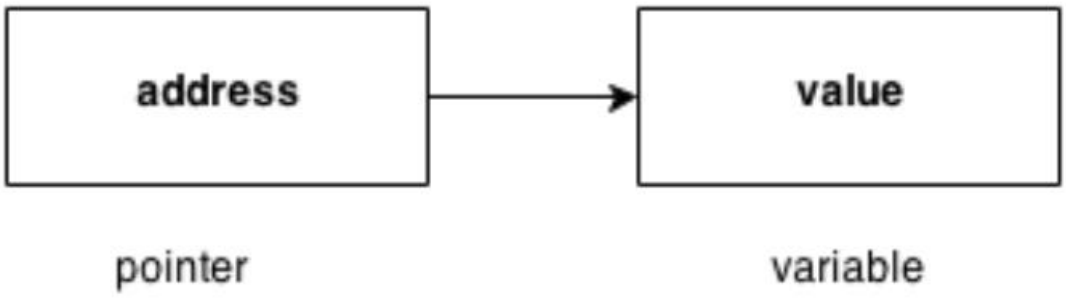
INTRODUCTION TO PROGRAMMING

UNIT-4

Pointers

Definition:

Pointer is a variable that stores/hold address of another variable of same data type/ t is also known as locator or indicator that points to an address of a value. A pointer is a derived data type in C



Benefit of using pointers

- Pointers are more efficient in handling Array and Structure.
- Pointer allows references to function and thereby helps in passing of function as arguments to other function.
- It reduces length and the program execution time.
- It allows C to support dynamic memory management.

Declaration of Pointer

data_type* pointer_variable_name;

int* p;

Note: void type pointer works with all data types, but isn't used often.

Initialization of Pointer variable

Pointer Initialization is the process of assigning address of a variable to pointer variable. Pointer variable contains address of variable of same data type

int a = 10 ;

int *ptr ; //pointer declaration

ptr = &a ; //pointer initialization

or,

int *ptr = &a ; //initialization and declaration together

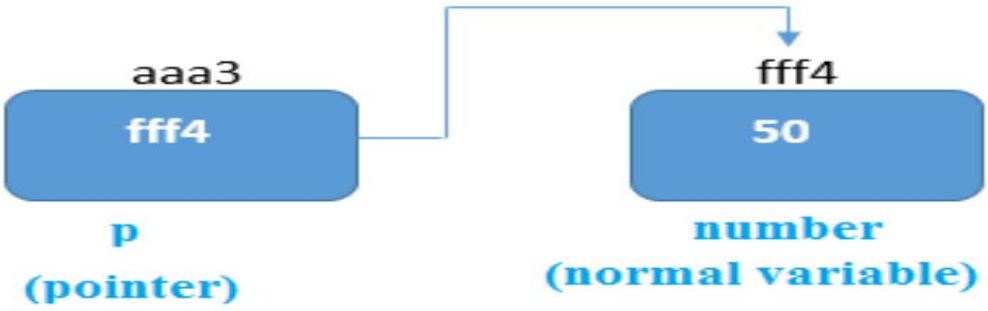
Note: Pointer variable always points to same type of data.

float a;

int *ptr;

ptr = &a; //ERROR, type mismatch

Above statement defines, p as pointer variable of type int. Pointer example



As you can see in the above figure, pointer variable stores the address of number variable i.e. fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.

By the help of * (indirection operator), we can print the value of pointer variable p.

Reference operator (&) and Dereference operator (*)

& is called reference operator. It gives you the address of a variable. There is another operator that gets you the value from the address, it is called a dereference operator (*).

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	address of operator	determines the address of a variable.
* (asterisk sign)	indirection operator	accesses the value at the address.

Dereferencing of Pointer

Once a pointer has been assigned the address of a variable. To access the value of variable, pointer is dereferenced, using the indirection operator *.

```
int a,*p;

a = 10;

p = &a;

printf("%d",*p); //this will print the value of a.

printf("%d",*&a); //this will also print the value of a.

printf("%u",&a); //this will print the address of a.

printf("%u",p); //this will also print the address of a.

printf("%u",&p); //this will also print the address of p.
```

KEY POINTS TO REMEMBER ABOUT POINTERS IN C:

- Normal variable stores the value whereas pointer variable stores the address of the variable.
- The content of the C pointer always be a whole number i.e. address.
- Always C pointer is initialized to null, i.e. int *p = null.The value of null pointer is 0.
- & symbol is used to get the address of the variable.
- Symbol is used to get the value of the variable that the pointer is pointing to.
- If a pointer in C is assigned to NULL, it means it is pointing to nothing.
- Two pointers can be subtracted to know how many elements are available between these two pointers.
- But, Pointer addition, multiplication, division are not allowed.
- The size of any pointer is 2 byte (for 16 bit compiler).

Example:

```
#include <stdio.h>

#include <conio.h>
```

```
void main()
{
int number=50;
int *p;
clrscr();
p=&number; //stores the address of number variable
printf("Address of number variable is %x \n",&number);
printf("Address of p variable is %x \n",p);
printf("Value of p variable is %d \n",*p);
getch();
}
```

Output

```
Address of number variable is fff4
Address of p variable is fff4
Value of p variable is 50
```

Example:

```
#include <stdio.h>
int main()
{
int *ptr, q;
q = 50;
/* address of q is assigned to ptr */
ptr = &q;
/* display q's value using ptr variable */
printf("%d", *ptr);
```

```
return 0;
}
```

Output

50

Example:

```
#include <stdio.h>
int main()
{
 int var =10;
 int *p;
 p= &var;
 printf ( "\n Address of var is: %u", &var);
 printf ( "\n Address of var is: %u", p);
 printf ( "\n Address of pointer p is: %u", &p);
 /* Note I have used %u for p's value as it should be an address*/
 printf( "\n Value of pointer p is: %u", p);
 printf ( "\n Value of var is: %d", var);
 printf ( "\n Value of var is: %d", *p);
 printf ( "\n Value of var is: %d", *( &var));
}
```

Output:

```
Address of var is: 00XBBA77
Address of var is: 00XBBA77
Address of pointer p is: 77221111
Value of pointer p is: 00XBBA77
```

Value of var is: 10

Value of var is: 10

Value of var is: 10

NULL Pointer

A pointer that is not assigned any value but NULL is known as NULL pointer. If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value.

Or

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a null pointer.int *p=NULL;

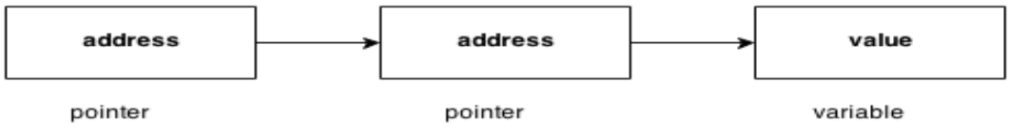
Note: The NULL pointer is a constant with a value of zero defined in several standard libraries/ in most the libraries, the value of pointer is 0 (zero)

Example:

The value of ptr is 0

Pointers to Pointers

Pointers can point to other pointers /pointer refers to the address of another pointer. Pointer can point to the address of another pointer which points to the address of a value.

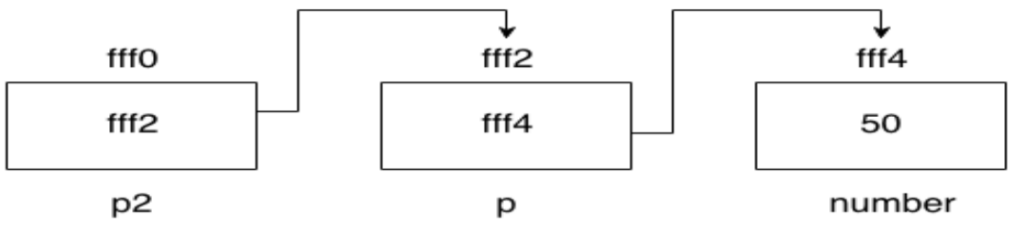


syntax of pointer to pointer

```
int **p2;
```

pointer to pointer example

Let's see an example where one pointer points to the address of another pointer.



Example:

```
#include <stdio.h>
#include <conio.h>
void main(){
int number=50;
int *p; //pointer to int
int **p2; //pointer to pointer
clrscr();
p=&number; //stores the address of number variable
p2=&p;
printf("Address of number variable is %x \n",&number);
printf("Address of p variable is %x \n",p);
printf("Value of *p variable is %d \n",*p);
printf("Address of p2 variable is %x \n",p2);
printf("Value of **p2 variable is %d \n",**p);
getch();
}
```

Output

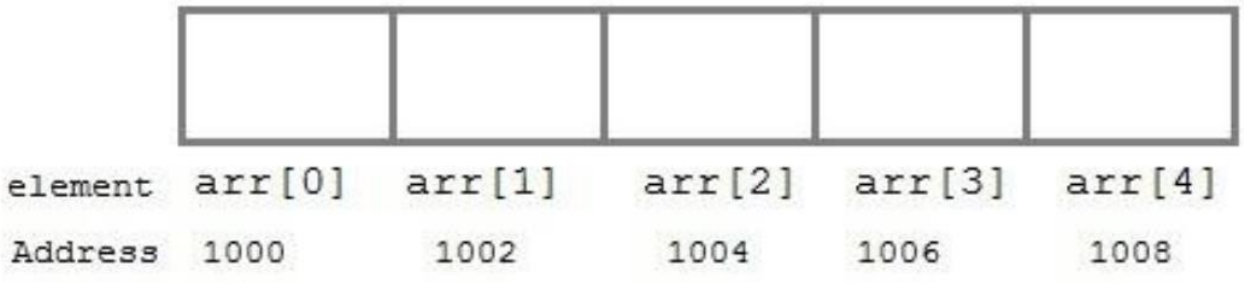
```
Address of number variable is fff4
Address of p variable is fff4
Value of *p variable is 50
Address of p2 variable is fff2
Value of **p variable is 50
```

Arrays and Pointers

When an array is declared, compiler allocates sufficient amount of memory to contain all the elements of the array. Base address which gives location of the first element is also allocated by the compiler.

Suppose we declare an array arr, int arr[5]={ 1, 2, 3, 4, 5 };

Assuming that the base address of arr is 1000 and each integer requires two byte, the five element will be stored as follows



Here variable arr will give the base address, which is a constant pointer pointing to the element, arr[0]. Therefore arr is containing the address of arr[0] i.e 1000.

arr is equal to &arr[0] // by default

We can declare a pointer of type int to point to the array arr.

```
int arr[5]={ 1, 2, 3, 4, 5 };

int *p;

p = arr;
```

or

```
p = &arr[0];  //both the statements are equivalent.
```

Now we can access every element of array arr using p++ to move from one element to another.

NOTE : You cannot decrement a pointer once incremented. p-- won't work.

Pointer to Array

we can use a pointer to point to an Array, and then we can use that pointer to access the array.

Lets have an example,

```
int i;

int a[5] = {1, 2, 3, 4, 5};

int *p = a; // same as int*p = &a[0]

for (i=0; i<5; i++)

{
```

```
printf("%d", *p);

p++;

}
```

In the above program, the pointer *p will print all the values stored in the array one by one. We can also use the Base address (a in above case) to act as pointer and print all the values.

Replacing the **printf("%d", *p);** statement of above example, with below mentioned statements. Lets see what will be the result.

printf("%d", a[i]); ⟶ **prints the array, by incrementing index**

printf("%d", i[a]); ⟶ **this will also print elements of array**

printf("%d", a+i); ⟶ **This will print address of all the array elements**

printf("%d", *(a+i)); ⟶ **Will print value of array element.**

printf("%d", *a); ⟶ **will print value of a[0] only**

a++; ⟶ **Compile time error, we cannot change base address of the array.**

Relation between Arrays and Pointers

Consider an array:

int arr[4];

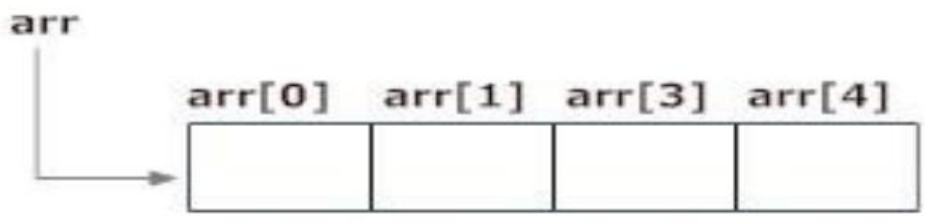


Figure: Array as Pointer

In C programming, name of the array always points to address of the first element of an array.

In the above example, arr and &arr[0] points to the address of the first element.

&arr[0] is equivalent to **arr**

Since, the addresses of both are the same, the values of arr and &arr[0] are also the same.

arr[0] is equivalent to *arr (value of an address of the pointer)

Similarly,

&arr[1] is equivalent to (arr + 1) AND, arr[1] is equivalent to *(arr + 1).

&arr[2] is equivalent to (arr + 2) AND, arr[2] is equivalent to *(arr + 2).

&arr[3] is equivalent to (arr + 3) AND, arr[3] is equivalent to *(arr + 3).

&arr[i] is equivalent to (arr + i) AND, arr[i] is equivalent to *(arr + i).

Example: Program to find the sum of six numbers with arrays and pointers

```
#include <stdio.h>
int main()
{
 int i, classes[6],sum = 0;
 printf("Enter 6 numbers:\n");
 for(i = 0; i < 6; ++i)
 {
 // (classes + i) is equivalent to &classes[i]
 scanf("%d",(classes + i));
 // *(classes + i) is equivalent to classes[i]
 sum += *(classes + i);
 }
 printf("Sum = %d", sum);
 return 0;
```

}

Output

Enter 6 numbers:

2

3

4

5

3

4

Sum = 21

Pointer Arithmetic and Arrays

Pointer holds address of a value, so there can be arithmetic operations on the pointer variable.

There are four arithmetic operators that can be used on pointers:

- Increment(++)
- Decrement(--)
- Addition(+)
- Subtraction(-)

Increment pointer:

1. Incrementing Pointer is generally used in array because we have contiguous memory in array and we know the contents of next memory location.

2. Incrementing Pointer Variable Depends Upon data type of the Pointer variable.

The formula of incrementing pointer is given below:

new_address= current_address + i * size_of(data type)

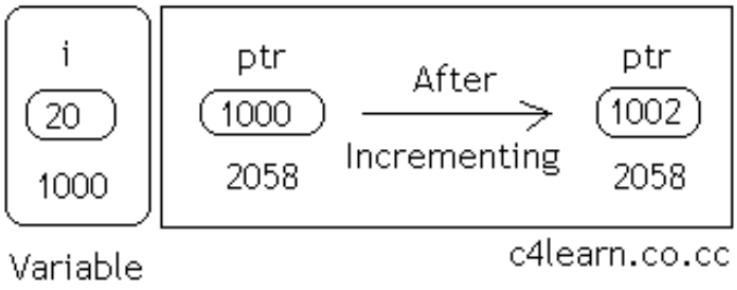
Three rules should be used to increment pointer

Address + 1 = Address

Address++ = Address

++Address = Address

Pictorial Representation :



Data Type	Older Address stored in pointer	Next Address stored in pointer after incrementing (ptr++)
int	1000	1002
float	1000	1004
char	1000	1001

Note:

32 bit

For 32 bit int variable, it will increment to 2 byte.

64 bit

For 64 bit int variable, it will increment to 4 byte.

Example:

```
#include <stdio.h>

void main()

{

int number=50;
```

```
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+1;
printf("After increment: Address of p variable is %u \n",p);
}
```

Output

```
Address of p variable is 3214864300
After increment: Address of p variable is 3214864304
```

Decrement(--)

Like increment, we can decrement a pointer variable.

formula of decrementing pointer

new_address= current_address - i * size_of(data type)

Example:

```
#include <stdio.h>
void main()
{
int number=50;
int *p; //pointer to int
p=&number; //stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p-1;
printf("After decrement: Address of p variable is %u \n",p);
}
```

Output

Address of p variable is 3214864300

After decrement: Address of p variable is 3214864296

Addition(+)

We can add a value to the pointer variable.

formula of adding value to pointer

new_address= current_address + (number * size_of(data type))

Note:

32 bit

For 32 bit int variable, it will add 2 * number.

64 bit

For 64 bit int variable, it will add 4 * number.

Example:

```
#include <stdio.h>
void main()
{
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+3; //adding 3 to pointer variable
printf("After adding 3: Address of p variable is %u \n",p);
}
```

Output

Address of p variable is 3214864300

After adding 3: Address of p variable is 3214864312

Subtraction (-)

Like pointer addition, we can subtract a value from the pointer variable. The formula of subtracting value from pointer variable.

new_address= current_address - (number * size_of(data type))

Example:

```
#include <stdio.h>

void main()

{

int number=50;

int *p;//pointer to int

p=&number;//stores the address of number variable

printf("Address of p variable is %u \n",p);

p=p-3; //subtracting 3 from pointer variable

printf("After subtracting 3: Address of p variable is %u \n",p);

}
```

Output

Address of p variable is 3214864300

After subtracting 3: Address of p variable is 3214864288

Array of Pointers

An array of pointers would be an array that holds memory locations. An array of pointers is an indexed set of variables in which the variables are pointers (a reference to a location in memory).

Syntax:

data_type_name * variable name

Example

```
int *ptr[MAX];
```

Array alpha[]	Pointer *a*
alpha[0]	*a
alpha[1]	*(a+1)
alpha[2]	*(a+2)
alpha[3]	*(a+3)
alpha[*n*]	*(a+*n*)

Example1:

```
#include <stdio.h>

const int MAX = 3;

int main ()

{

 int var[] = {10, 100, 200};

 int i, *ptr[MAX];

 for ( i = 0; i < MAX; i++)

{

ptr[i] = &var[i]; /* assign the address of integer. */

 }

 for ( i = 0; i < MAX; i++) {

 printf("Value of var[%d] = %d\n", i, *ptr[i] );

 }

 return 0;
```

```
}
```

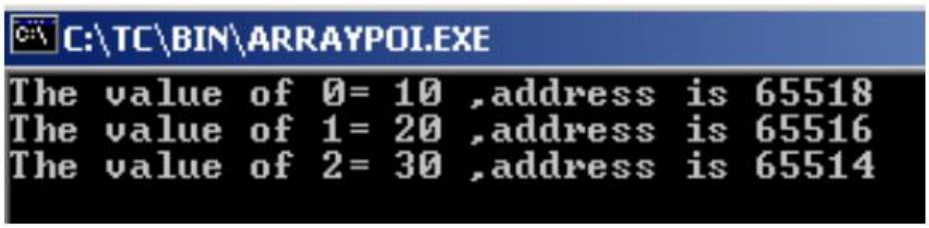
Output

```
Value of var[0] = 10

Value of var[1] = 100

Value of var[2] = 200
```

Example2:

```
#include <stdio.h>

#include <conio.h>

main()

{

clrscr();

int *array[3];

int x = 10, y = 20, z = 30;

int i;

array[0] = &x;

array[1] = &y;

array[2] = &z;

for (i=0; i< 3; i++) {

printf("The value of %d= %d ,address is %u\t \n", i, *(array[i]),

array[i]);

}

getch();

return 0;

}
```

Output



Structure Definition

Structure is a user defined data type which hold or store heterogeneous/different types data item

Or

Element in a single variable. It is a Combination of primitive and derived data type.

Or

A structure is a collection of one or more data items of different data types, grouped together under a single name.

Variables inside the structure are called members of structure. Each element of a structure is called a member.

Struct keyword is used to define/create a structure. Struct define a new data type which is a Collection of different type of data.

Syntax

struct structure_name /tag name

{

data_type member1;

data_type member2;

.

.

data_type member n;

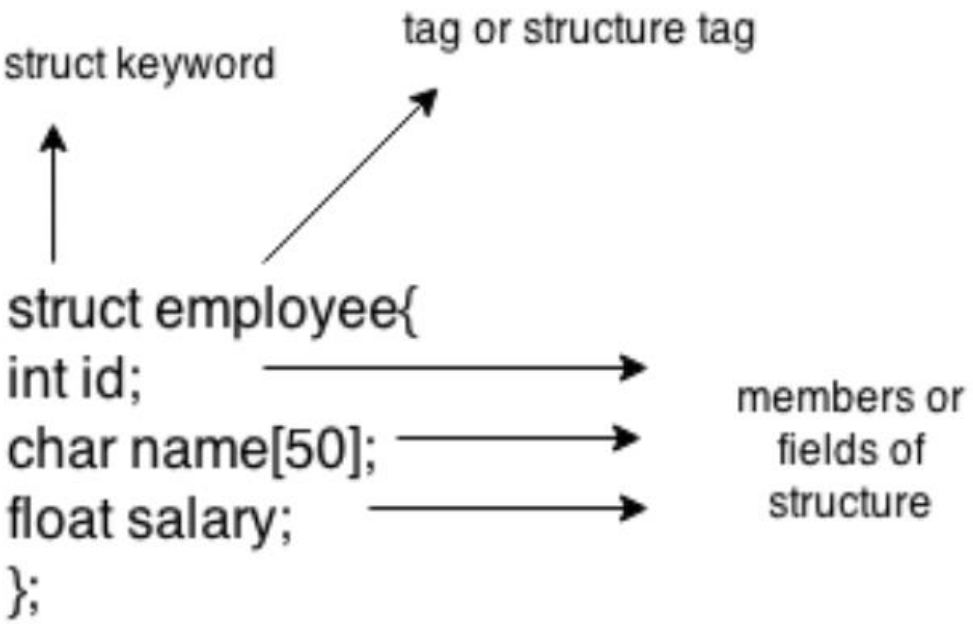
};

Note: Don't forget the semicolon }; in the ending line.

Example

```
struct employee
{ int id;
char name[50];
float salary;
};
```

Here, struct is the keyword, employee is the tag name of structure; id, name and salary are the members or fields of the structure. Let's understand it by the diagram given below:



Syntax to create structure variable

struct tagname/structure_name variable;

Declaring structure variable

We can declare variable for the structure, so that we can access the member of structure easily.

There are two ways to declare structure variable:

1. By struct keyword within main() function/ Declaring Structure variables separately

2. By declaring variable at the time of defining structure/ Declaring Structure Variables with Structure definition

1st way:

Let's see the example to declare structure variable by struct keyword. It should be declared

within the main function.

```
struct employee

{

int id;

char name[50];

float salary;

};
```

Now write given code inside the main() function.

```
struct employee e1, e2;
```

2nd way:

Let's see another way to declare variable at the time of defining structure.

```
struct employee

{ int id;

char name[50];

float salary;

}e1,e2;
```

Which approach is good?

But if no. of variable are not fixed, use 1st approach. It provides you flexibility to declare the structure variable many times.

If no. of variables are fixed, use 2nd approach. It saves your code to declare variable in main() function.

Structure Initialization

structure variable can also be initialized at compile time.

```
struct Patient

{

float height;
```

int weight;

int age;

};

struct Patient p1 = { 180.75 , 73, 23 }; **//initialization**

or

struct patient p1;

p1.height = 180.75; **//initialization of each member separately**

p1.weight = 73;

p1.age = 23;

Accessing Structures/ Accessing members of structure

There are two ways to access structure members:

1. By . (member or dot operator)

2. By -> (structure pointer operator)

When the variable is normal type then go for struct to member operator.

When the variable is pointer type then go for pointer to member operator.

Any member of a structure can be accessed as:

structure_variable_name.member_name

Example

struct book

{

char name[20];

char author[20];

int pages;

};

struct book b1;

for accessing the structure members from the above example

b1.name, b1.author, b1.pages:

Example

```
struct emp
{
int id;
char name[36];
int sal;
};
sizeof(struct emp) // --> 40 byte (2byte+36byte+2byte)
```

Example of Structure in C

```
#include<stdio.h>
#include<conio.h>
struct emp
{
int id;
char name[36];
float sal;
};
void main()
{
struct emp e;
clrscr();
printf("Enter employee Id, Name, Salary: ");
scanf("%d",&e.id);
```

```
scanf("%s",&e.name);

scanf("%f",&e.sal);

printf("Id: %d",e.id);

printf("\nName: %s",e.name);

printf("\nSalary: %f",e.sal);

getch();
```

Output

Output: Enter employee Id, Name, Salary: 5 Spidy 45000

Id : 05

Name: Spidy

Salary: 45000.00

Example

```
#include <stdio.h>

#include <string.h>

struct employee

{ int id;

char name[50];

}e1; //declaring e1 variable for structure

int main( )

{

//store first employee information

e1.id=101;

strcpy(e1.name, "Sonoo Jaiswal");  //copying string into char array

//printing first employee information

printf( "employee 1 id : %d\n", e1.id);
```

printf("employee 1 name : %s\n", e1.name);

return 0;

}

Output:

employee 1 id : 101

employee 1 name : Sonoo Jaiswal

Difference Between Array and Structure

1	Array is collection of homogeneous data.	Structure is the collection of heterogeneous data.
2	Array data are access using index.	Structure elements are access using . operator.
3	Array allocates static memory.	Structures allocate dynamic memory.
4	Array element access takes less time than structures.	Structure elements takes more time than Array.

Arrays of Structures

Array of structures to store much information of different data types. Each element of the array representing a structure variable. The array of structures is also known as collection of structures.

Ex : if you want to handle more records within one structure, we need not specify the number of structure variable. Simply we can use array of structure variable to store them in one structure variable.

Example : struct employee emp[5];

Example of structure with array that stores information of 5 students and prints it.

#include<stdio.h>

#include<conio.h>

#include<string.h>

struct student{

int rollno;

char name[10];

```
};
void main(){
int i;
struct student st[5];
clrscr();
printf("Enter Records of 5 students");
for(i=0;i<5;i++){
printf("\nEnter Rollno:");
scanf("%d",&st[i].rollno);
printf("\nEnter Name:");
scanf("%s",&st[i].name);
}
printf("\nStudent Information List:");
for(i=0;i<5;i++){
printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
getch();
}
```

Output:

```
Enter Records of 5 students
Enter Rollno:1
Enter Name:Sonoo
Enter Rollno:2
Enter Name:Ratan
Enter Rollno:3
```

Enter Name:Vimal

Enter Rollno:4

Enter Name:James

Enter Rollno:5

Enter Name:Sarfraz

Student Information List:

Rollno:1, Name:Sonoo

Rollno:2, Name:Ratan

Rollno:3, Name:Vimal

Rollno:4, Name:James

Rollno:5, Name:Sarfraz

Passing Structures through Pointers

Example

```
#include <stdio.h>
#include <string.h>
struct student
{
int id;
char name[30];
float percentage;
};
int main()
{
int i;
struct student record1 = {1, "Raju", 90.5};
```

```
struct student *ptr;

ptr = &record1;

printf("Records of STUDENT1: \n");

printf(" Id is: %d \n", ptr->id);

printf(" Name is: %s \n", ptr->name);

printf(" Percentage is: %f \n\n", ptr->percentage);

return 0;

}
```

OUTPUT:

```
Records of STUDENT1:

Id is: 1

Name is: Raju

Percentage is: 90.500000
```

Self-referential Structures

A structure consists of at least a pointer member pointing to the same structure is known as a self-referential structure. A self referential structure is used to create data structures like linked lists, stacks, etc. Following is an example of this kind of structure:

A self-referential structure is one of the data structures which refer to the pointer to (points) to another structure of the same type. For example, a linked list is supposed to be a self-referential data structure. The next node of a node is being pointed, which is of the same struct type. For

example,

```
Syntax : struct tag_name

 {

 type member1;

 type membere2;

 : :

 : :
```

typeN memberN;

struct tag_name *name;

}

Where *name refers to the name of a pointer variable.

Ex:

struct emp

{

int code;

struct emp *name;

}

Unions

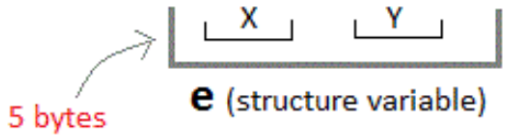
A union is a special data type available in C that allows to store different data types in the same memory location.

Unions are conceptually similar to structures. The syntax of union is also similar to that of structure. The only difference is in terms of storage. In structure each member has its own storage location, whereas all members of union use a single shared memory location which is equal to the size of its largest data member.

We can access only one member of union at a time. We can't access all member values at the same time in union. But, structure can access all member values at the same time. This is because, Union allocates one common storage space for all its members. Whereas Structure allocates storage space for all its members separately.

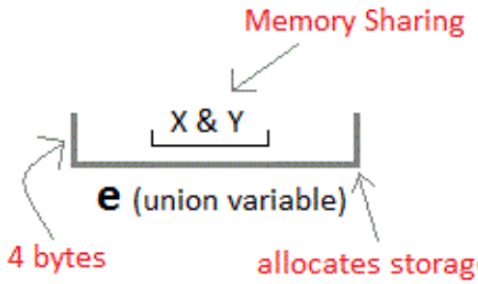
Structure

```
struct Emp
{
 char X ;     // size 1 byte
 float Y ;    // size 4 byte
} e ;
```



Unions

```
union Emp
{
 char X ;
 float Y ;
} e ;
```



Structure

```
struct Employee{
char x; // size 1 byte
int y; //size 2 byte
float z; //size 4 byte
}e1; //size of e1 = 7 byte
```

size of e1= 1 + 2 + 4 = 7

Union

```
union Employee{
char x; // size 1 byte
int y; //size 2 byte
float z; //size 4 byte
}e1; //size of e1 = 4 byte
```

size of e1= 4 (maximum size of 1 element)

syntax

union union_name

{

data_type member1;

data_type member2;

.

.

```
data_type memeberN;
};
```

Example

```
union employee
{ int id;
char name[50];
float salary;
};
```

Example

```
#include <stdio.h>
#include <string.h>

union Data
{
   int i;
   float f;
   char str[20];
};
int main( )
{
   union Data data;
   data.i = 10;
   data.f = 220.5;
   strcpy( data.str, "C Programming");
   printf( "data.i : %d\n", data.i);
```

```
   printf( "data.f : %f\n", data.f);

   printf( "data.str : %s\n", data.str);

   return 0;

}
```

When the above code is compiled and executed, it produces the following result −

OUTPUT:

```
data.i : 1917853763

data.f : 4122360580327794860452759994368.000000

data.str : C Programming
```

Here, we can see that the values of i and f members of union got corrupted because the final value assigned to the variable has occupied the memory location and this is the reason that the value of str member is getting printed very well.

Now let's look into the same example once again where we will use one variable at a time which is the main purpose of having unions.

```
#include <stdio.h>

#include <string.h>

union Data

{

   int i;

   float f;

   char str[20];

};

int main( )

{

   union Data data;
```

```
   data.i = 10;
   printf( "data.i : %d\n", data.i);
   data.f = 220.5;
   printf( "data.f : %f\n", data.f);
   strcpy( data.str, "C Programming");
   printf( "data.str : %s\n", data.str);
   return 0;
}
```

When the above code is compiled and executed, it produces the following result −

OUTPUT:

```
data.i : 10


data.f : 220.500000
data.str : C Programming
```

Here, all the members are getting printed very well because one member is being used at a time.

Difference between Structure and Union

	Structure	Union
1	For defining structure use struct keyword.	For defining union we use union keyword
2	Structure occupies more memory space than union.	Union occupies less memory space than Structure.
3	In Structure we can access all members of structure at a time.	In union we can access only one member of union at a time.
4	Structure allocates separate storage space for its every members.	Union allocates one common storage space for its all members. Union find which member need more memory than other member, then it allocate that much space

Enumerations

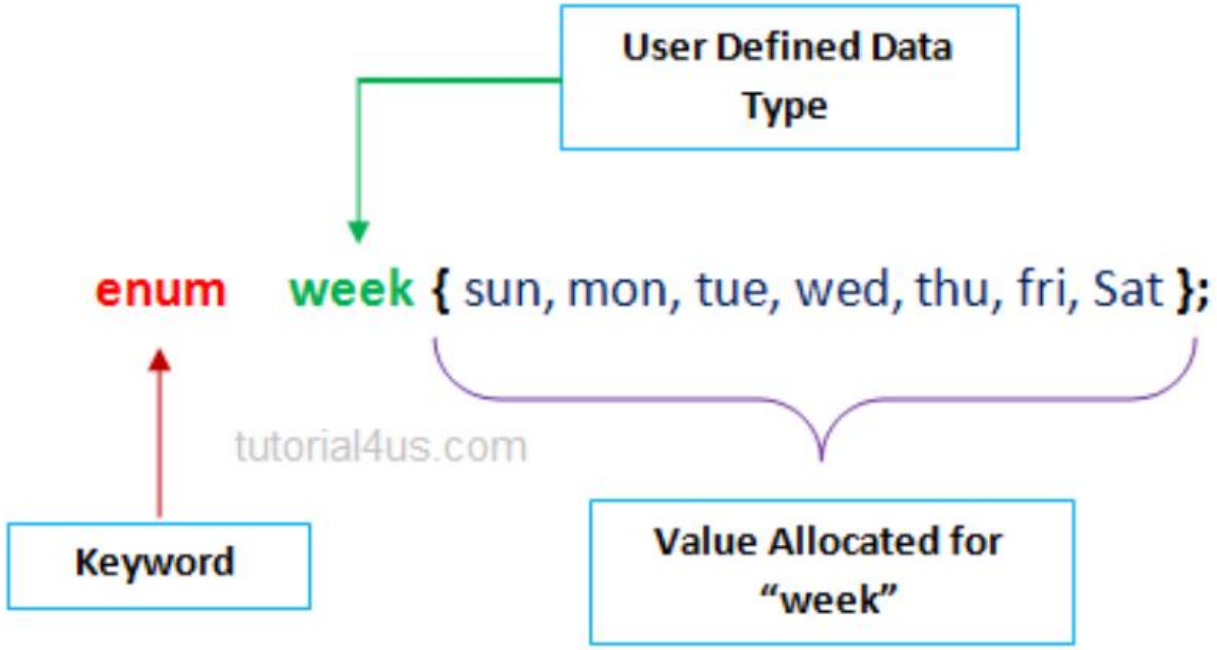
An enum is a keyword, it is an user defined data type. All properties of integer are applied on Enumeration data type so size of the enumerator data type is 2 byte. It work like the Integer.

It is used for creating an user defined data type of integer. Using enum we can create sequence of integer constant value.

Syntax

enum tagname {value1, value2, value3,....};

- In above syntax enum is a keyword. It is a user defiend data type.
- In above syntax tagname is our own variable. tagname is any variable name.
- value1, value2, value3,.... are create set of enum values.



It is start with 0 (zero) by default and value is incremented by 1 for the sequential identifiers in the list. If constant one value is not initialized then by default sequence will be start from zero and next to generated value should be previous constant value one.

Example of Enumeration in C

```
#include<stdio.h>

#include<conio.h>

enum ABC {x,y,z};

void main()

{

int a;

clrscr();

a=x+y+z; //0+1+2

printf("Sum: %d",a);

getch();

}
```

Output

Sum: 3

Example of Enumeration in C

```
#include<stdio.h>
#include<conio.h>
enum week {sun, mon, tue, wed, thu, fri, sat};
void main()
{
enum week today;
today=tue;
printf("%d day",today+1);
getch();
}
```

Output

3 day

typedef

The typedef is a keyword that allows the programmer to create a new data type name for an existing data type. So, the purpose of typedef is to redefine the name of an existing variable type.

Syntax

```
typedef datatype alias_name;
```

Example of typedef

```
#include<stdio.h>
#include<conio.h>
typedef int Intdata;  // Intdata is alias name of int
void main()
{
```

```
int a=10;

Integerdata b=20;

typedef Intdata Integerdata; // Integerdata is again alias name of Intdata

Integerdata s;

clrscr();

s=a+b;

printf("\n Sum:= %d",s);

getch();

}
```

Output

Sum: 20

Dynamic Memory Allocation Functions

The concept of dynamic memory allocation in c language enables the C programmer to allocate memory at runtime.

Or

The process of allocating memory at runtime is known as dynamic memory allocation. Library routines known as "memory management functions" are used for allocating and freeing memory during execution of a program. These functions are defined in stdlib.h.Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

1. malloc()

2. calloc()

3. realloc()

4. free()

Difference between static memory allocation and dynamic memory allocation.

static memory allocation	dynamic memory allocation
memory is allocated at compile time.	memory is allocated at run time.
memory can't be increased while executing program.	memory can be increased while executing program.
used in array.	used in linked list.

Methods used for dynamic memory allocation.

malloc()	allocates single block of requested memory.
calloc()	allocates multiple block of requested memory.
realloc()	reallocates the memory occupied by malloc() or calloc() functions.
free()	frees the dynamically allocated memory.

Note: Dynamic memory allocation related function can be applied for any data type that's why dynamic memory allocation related functions return void*.

Memory Allocation Process

Global variables, static variables and program instructions get their memory in permanent storage area whereas local variables are stored in area called Stack. The memory space between these two region is known as Heap area. This region is used for dynamic memory allocation during execution of the program. The size of heap keep changing.

malloc()

malloc stands for "memory allocation".

The malloc() function allocates single block of requested memory at runtime. This function reserves a block of memory of given size and returns a pointer of type void. This means that we can assign it to any type of pointer using typecasting. It doesn't initialize memory at execution time, so it has garbage value initially. If it fails to locate enough space (memory) it returns a NULL pointer.

syntax

ptr=(cast-type*)malloc(byte-size)

Example

```
int *x;

x = (int*)malloc(100 * sizeof(int)); //memory space allocated to variable x

free(x); //releases the memory allocated to variable x
```

This statement will allocate either 200 or 400 according to size of int 2 or 4 bytes respectively and the pointer points to the address of first byte of memory.

Example

```
#include <stdio.h>

#include <stdlib.h>

int main()

{

 int num, i, *ptr, sum = 0;

 printf("Enter number of elements: ");

 scanf("%d", &num);

 ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc

 if(ptr == NULL)

 {

 printf("Error! memory not allocated.");

 exit(0);

 }

 printf("Enter elements of array: ");

 for(i = 0; i < num; ++i)

 {

 scanf("%d", ptr + i);
```

```
 sum += *(ptr + i);
 }
 printf("Sum = %d", sum);
 free(ptr);
 return 0;
}
```

calloc()

calloc stands for "contiguous allocation".

Calloc() is another memory allocation function that is used for allocating memory at runtime. calloc function is normally used for allocating memory to derived data types such as arrays and structures. The calloc() function allocates multiple block of requested memory.It initially initialize (sets) all bytes to zero.If it fails to locate enough space(memory) it returns a NULL pointer.

The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size.

Syntax

ptr = (cast-type*)calloc(n/number, element-size);

calloc() required 2 arguments of type count, size-type.

Count will provide number of elements; size-type is data type size

Example

```
int*arr;
arr=(int*)calloc(10, sizeof(int)); // 20 byte
cahr*str;
str=(char*)calloc(50, siceof(char)); // 50 byte
```

Example

```
struct employee
{
char *name;
```

```
int salary;
};
typedef struct employee emp;
emp *e1;
e1 = (emp*)calloc(30,sizeof(emp));
```

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
 int num, i, *ptr, sum = 0;
 printf("Enter number of elements: ");
 scanf("%d", &num);
 ptr = (int*) calloc(num, sizeof(int));
 if(ptr == NULL)
 {
 printf("Error! memory not allocated.");
 exit(0);
 }
 printf("Enter elements of array: ");
 for(i = 0; i < num; ++i)
 {
 scanf("%d", ptr + i);
 sum += *(ptr + i);
 }
```

```
 printf("Sum = %d", sum);

 free(ptr);

 return 0;

}
```

Diffrence between malloc() and calloc()

calloc()	**malloc()**
calloc() initializes the allocated memory with 0 value.	malloc() initializes the allocated memory with garbage values.
Number of arguments is 2	Number of argument is 1
Syntax : (cast_type *)calloc(blocks , size_of_block);	**Syntax :** (cast_type *)malloc(Size_in_bytes);

realloc(): changes memory size that is already allocated to a variable.

Or

If the previously allocated memory is insufficient or more than required, you can change the previously allocated memory size using realloc().

If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function. In short, it changes the memory size. By using realloc() we can create the memory dynamically at middle stage. Generally by using realloc() we can reallocation the memory. Realloc() required 2 arguments of type void*, size_type.

Void* will indicates previous block base address, size-type is data type size. Realloc() will creates the memory in bytes format and initial value is garbage.

syntax

ptr=realloc(ptr, new-size)

Example

```
int *x;

x=(int*)malloc(50 * sizeof(int));
```

```
x=(int*)realloc(x,100); //allocated a new memory to variable x
```

Example

```
void*realloc(void*, size-type);
int *arr;
arr=(int*)calloc(5, sizeof(int));
.....
.....
....
arr=(int*)realloc(arr,sizeof(int)*10);
```

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
 int *ptr, i , n1, n2;
 printf("Enter size of array: ");
 scanf("%d", &n1);
 ptr = (int*) malloc(n1 * sizeof(int));
 printf("Address of previously allocated memory: ");
 for(i = 0; i < n1; ++i)
 printf("%u\t",ptr + i);
 printf("\nEnter new size of array: ");
 scanf("%d", &n2);
 ptr = realloc(ptr, n2);
 for(i = 0; i < n2; ++i)
```

```
 printf("%u\t", ptr + i);
 return 0;
}
```

free()

When your program comes out, operating system automatically release all the memory allocated by your program but as a good practice when you are not in need of memory anymore then you should release that memory by calling the function free().

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Or

Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on its own. You must explicitly use free() to release the space.

Syntax:

```
free(ptr);
```

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
 int num, i, *ptr, sum = 0;
 printf("Enter number of elements: ");
 scanf("%d", &num);
 ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
 if(ptr == NULL)
 {
 printf("Error! memory not allocated.");
```

```
 exit(0);
 }
 printf("Enter elements of array: ");
 for(i = 0; i < num; ++i)
 {
 scanf("%d", ptr + i);
sum += *(ptr + i);
 }
 printf("Sum = %d", sum);
 free(ptr);
 return 0;
}
```